

DISTRIBUTED JOB SCHEDULER

System Architecture & Implementation Submission

Candidate Name:	Methun-21
Submission Date:	July 5, 2026
GitHub Repository:	https://github.com/Methun-21/JobScheduler.git
Tech Stack:	Python (FastAPI), PostgreSQL, SQLAlchemy (asyncpg), React (Vite)

EXECUTIVE SUMMARY

This project implements a highly reliable, concurrent, and scalable distributed job scheduler platform. The system coordinates background worker processes using PostgreSQL database-level locks, eliminating double-execution errors without needing complex broker setups. It features multi-tenant organization boundaries, queue-level concurrency limits, multiple-parent DAG dependencies (with cycle protection), exponential backoff retries, and a premium glassmorphic dashboard built in React.

SYSTEM ARCHITECTURE & DESIGN

The architecture separates the Control Plane (FastAPI REST & WebSockets) from the Data Plane (background workers). Both communicate solely through PostgreSQL. The scheduler leader runs in a dedicated thread within the API process, obtaining ownership via a session-level advisory lock on a pinned connection, which dynamically evaluates cron template expressions and promotes scheduled events to active jobs.

Worker Concurrency & Claiming Model: To prevent concurrent execution of jobs when multiple workers poll simultaneously, the scheduler leverages transaction-level advisory locks combined with *SELECT FOR UPDATE SKIP LOCKED*. This guarantees that workers only claim jobs from active, unpaused queues while respecting queue-specific concurrency limits.

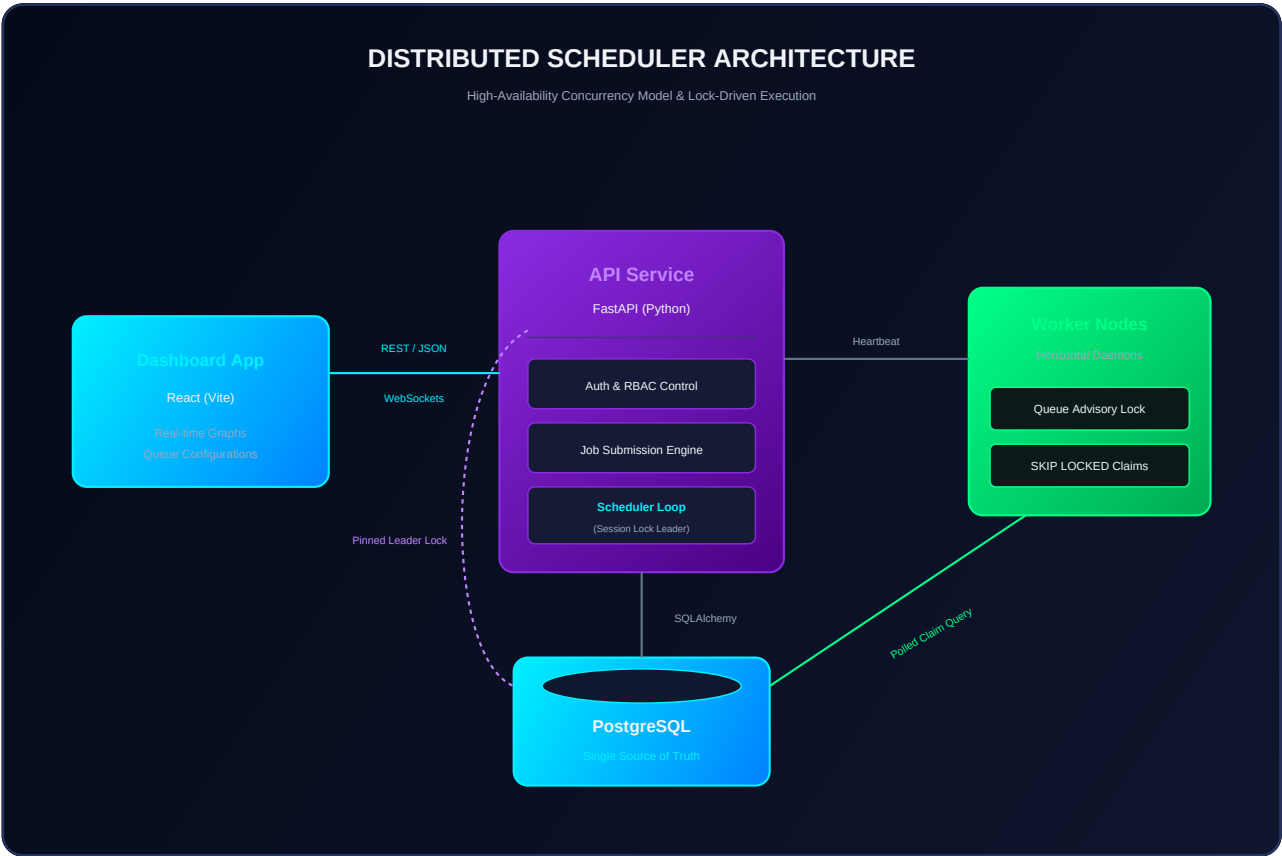


Figure 1: High-Level System Architecture and Connections

DATABASE SCHEMA & NORMALIZATION

The database is designed with 13 fully normalized tables ensuring strict multi-tenant access control and comprehensive auditing. Primary keys use UUIDs for global uniqueness, and execution histories are separated from job state definitions so that execution logs remain intact even if the master job record is archived or removed.

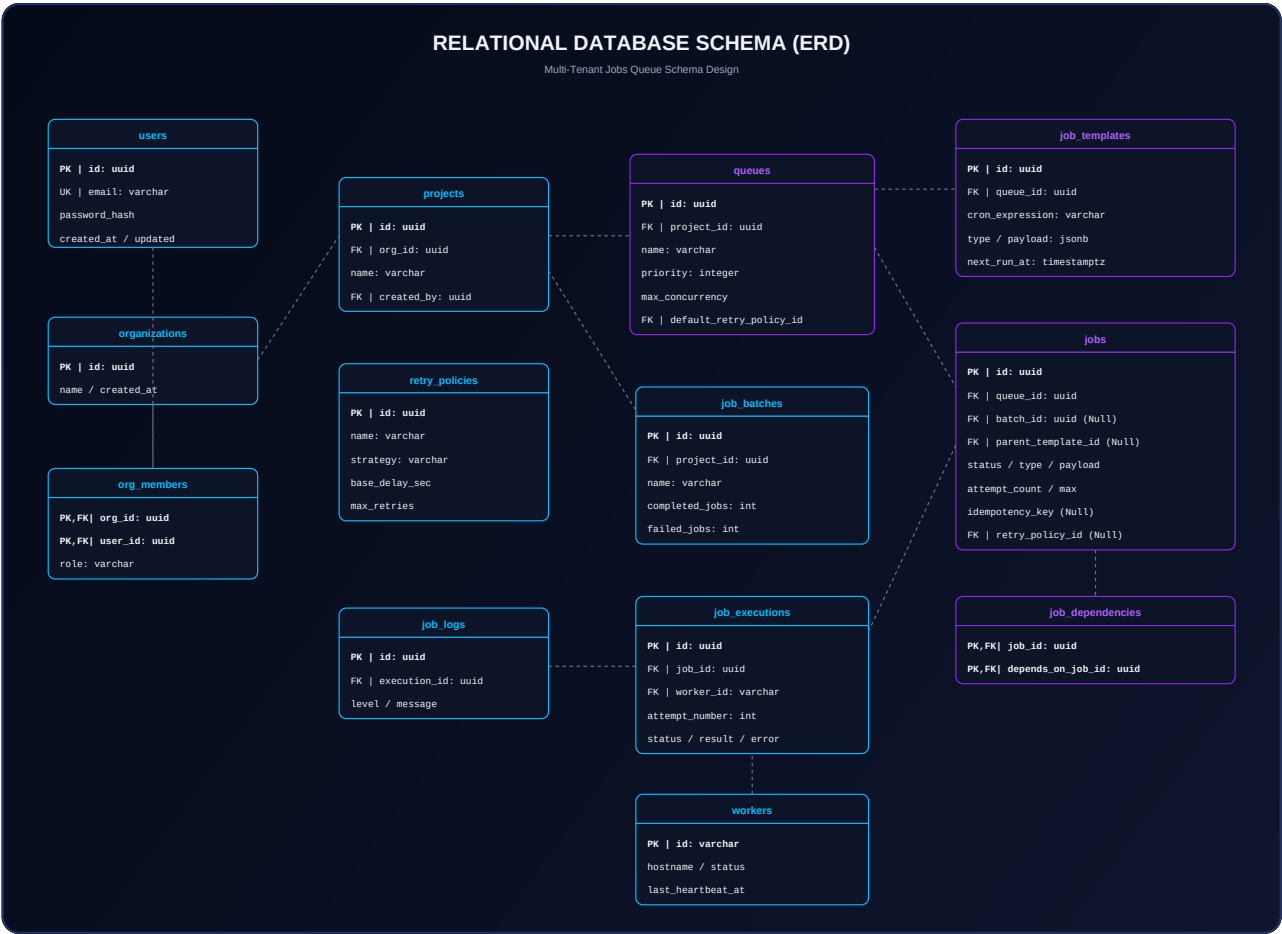
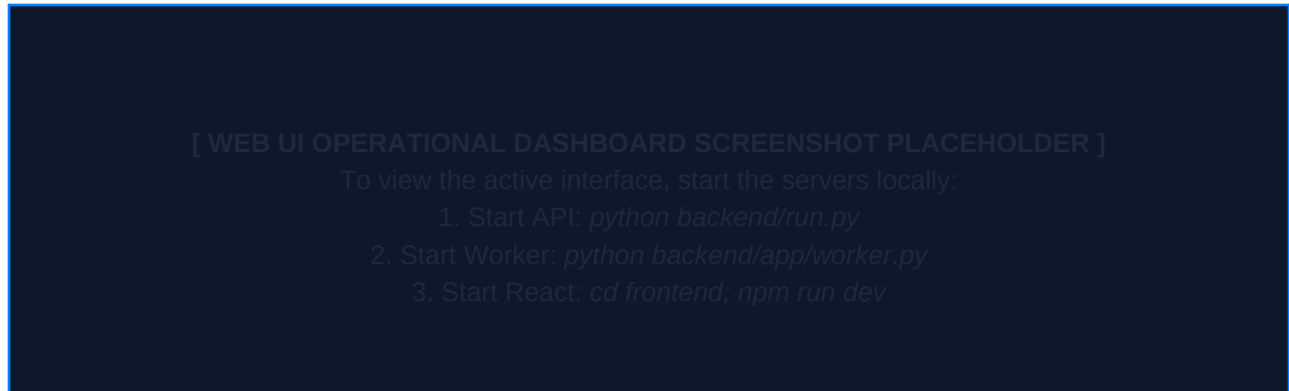


Figure 2: Relational Entity Relationship Diagram (ERD)

OPERATIONAL DEPLOYMENT & SCREENSHOTS

The React web client provides a high-fidelity glassmorphism interface displaying metric counters (workers active, jobs queued, completed, failed), queue details, template management, and an execution console. All updates stream live via WebSockets.

Operational Web UI Dashboard Placeholder:



RUNNING MANUAL TEST SUITE

To verify core properties (concurrency limits, DAG claims, DFS cycles, and backoff retries), run pytest:

```
python -m pytest backend/tests/
```

Submission Package Contents Checklist:

- Codebase repository on GitHub: <https://github.com/Methun-21/JobScheduler.git>
- System design keputusan: *design_decisions.md*
- Setup guides & Quick Start instructions: *README.md*
- Scalable architecture & DB graphics: *architecture_diagram.svg* & *er_diagram.svg*